Technical Architecture Document

Neutral Agent Pay by Vottun – Wirex

Enterprise x402 Payments SDK on Stellar

Document version	1.0
Date	July 2026
Prepared by	Vottun, S.L.
Partners	Wirex (lead partner, regulated fiat settlement) · DFNS (MPC wallet infrastructure)
Submission	SCF Build Award — Open Track · End-User Application
Networks	Stellar Testnet (`stellar:testnet`) → Stellar Mainnet (`stellar:pubnet`)

1. Purpose and scope

This document is the complete technical architecture for Neutral Agent Pay, submitted in support of an SCF Build Award application. It describes what will be built, how each component works, how the components interact on the Stellar network, which external services are relied upon, and which technical risks exist together with the mitigation designed for each.

It is written to be self-contained: an evaluator should be able to assess technical feasibility without consulting external material, while every non-obvious protocol claim is referenced to primary documentation in Section 15.

In scope: the x402 payment engine and SDK, the Soroban payment-control contracts, the DFNS wallet layer, the settlement path through an x402 facilitator, the indexing and reconciliation layer, the merchant dashboard, the Wirex fiat settlement integration, and the neutral gateway.

Out of scope: Wirex's internal ledger and card infrastructure, DFNS's internal MPC implementation, and any component of the Coinbase x402 specification itself. These are consumed as external dependencies with defined interfaces.

2. Executive summary

AI agents can call APIs but cannot pay for them. The x402 protocol — an open standard from the Coinbase Developer Platform — solves the protocol half of that problem by turning the HTTP `402 Payment Required` status code into a usable per-request payment challenge. On Stellar, x402 works through Soroban authorization, so a client pays for a request by signing an authorization entry rather than a full transaction.

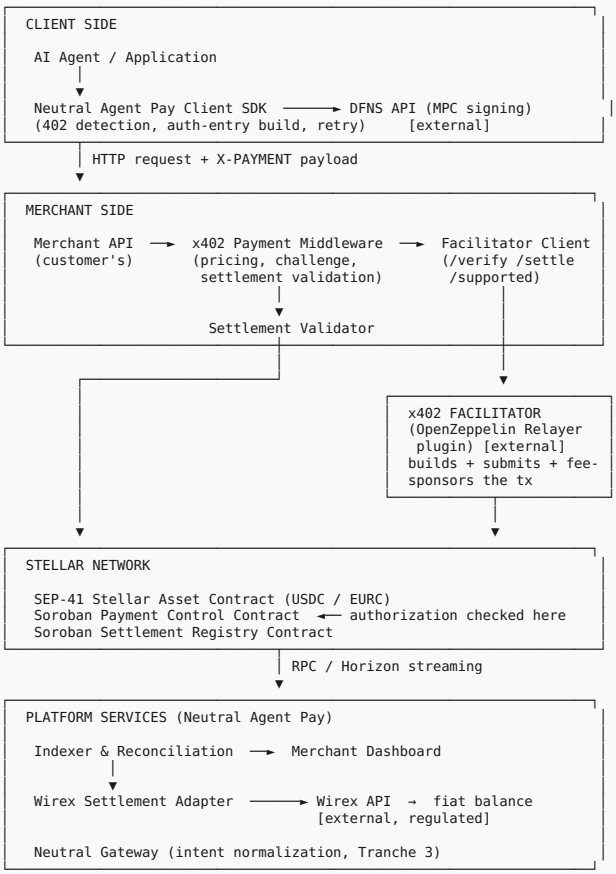
What does not exist yet is the operational layer a real business needs around that protocol: wallets that can be provisioned programmatically without a human clicking a browser extension, spending controls that bound what an autonomous agent can do, reconciliation that a finance team can audit, and a path from settled stablecoin to spendable fiat.

Neutral Agent Pay builds that layer on Stellar. A developer installs an SDK, declares which endpoints are paid and at what price, and is live. Behind that interface: DFNS MPC infrastructure provisions and signs from Stellar accounts programmatically; Soroban contracts enforce per-agent spending caps, endpoint allowlists and velocity limits; an x402 facilitator verifies and settles the payment on Stellar in USDC or EURC while sponsoring the network fee; an indexer reconciles every on-chain payment against the merchant ledger; and Wirex, a regulated payments entity already running live dual-stablecoin card settlement on Stellar, converts the settled balance into fiat.

The architecture deliberately consumes existing ecosystem infrastructure — the x402 facilitator, the SEP-41 Stellar Asset Contracts for USDC and EURC, OpenZeppelin's Relayer — rather than rebuilding it, and reuses the wallet management, transaction lifecycle, configuration and streaming components of the Wirex–Vottun–Stellar SDK that Vottun already delivered under a previous SCF award.

3. System overview

3.1 Component map



3.2 Component responsibility matrix

#	Component	Built by	Runs where	Stellar dependency	Tranche
C1	Client SDK (TypeScript)	Vottun	Agent / client process	Builds SEP-41 `transfer` invocation + auth entry	T1
C2	x402 Payment Middleware	Vottun	Merchant server	Issues 402 challenges with Stellar payment requirements	T1
C3	Facilitator Client	Vottun	Platform	Calls `/verify`, `/settle`, `/supported`	T1
C4	DFNS Wallet Adapter	Vottun (on DFNS)	Platform	Provisions Stellar accounts, trustlines, hash signing	T1
C5	Payment Control Contract	Vottun	Soroban	Enforces caps, allowlists, velocity at auth time	T2
C6	Settlement Registry Contract	Vottun	Soroban	Records settled payments, emits events	T2
C7	Indexer & Reconciliation	Vottun	Platform	Consumes Stellar RPC / Horizon streams	T2
C8	Merchant Dashboard	Vottun	Platform (web)	Reads indexer + on-chain state	T2
C9	Wirex Settlement Adapter	Vottun (on Wirex API)	Platform	Maps Stellar tx → fiat ledger entry	T2 (sandbox) / T3 (prod)
C10	Neutral Gateway	Vottun	Platform	Normalizes intents to Stellar settlement	T3
E1	x402 Facilitator	OpenZeppelin / SDF	External	Verifies, submits, fee-sponsors	consumed
E2	SEP-41 SAC (USDC, EURC)	Circle / SDF	Stellar	Token contract	consumed
E3	DFNS platform	DFNS	External	MPC key custody and signing	consumed
E4	Wirex platform	Wirex	External	Regulated fiat rails, KYB/KYC	consumed

Design principle: every box marked *external* is a dependency with a documented API and a defined fallback (Section 12). Nothing critical is a single point of failure without a stated alternative.

---

4. Protocol foundation: x402 on Stellar

Understanding the exact mechanics matters, because they determine what the wallet layer must be able to do and where the spending controls can be enforced.

4.1 The payment cycle

x402 is an open protocol for programmatic, per-request payments over HTTP, designed for AI agents and APIs. On Stellar it works with Soroban authorization so that clients pay for API requests via signed authorization entries — which is what makes it viable for micropayments.

Concretely, under the x402 v2 `exact` scheme on Stellar:

1. The client requests a protected resource and receives `402` with payment requirements: scheme, network identifier (`stellar:testnet` or `stellar:pubnet`), price, asset, and the destination address (`payTo`).
2. The client constructs a Soroban transaction whose operation is an `invokeHostFunction` calling `transfer` on the asset's SEP-41 contract, signs the **authorization entry** as the payer, and leaves the **transaction envelope unsigned**.
3. The client resubmits the original request with the payment payload attached.
4. The merchant middleware forwards the payload to a facilitator's `/verify` endpoint, and on success to `/settle`.
5. The facilitator rebuilds and submits the transaction on-chain, paying the network fee itself, and returns a settlement result.
6. The middleware serves the resource.

The two consequences that shape this architecture: **the payer never submits a transaction**, and **the payer's wallet must be able to sign Soroban authorization entries**, not just classic Stellar transactions. Stellar's own documentation lists auth-entry signing as the defining requirement for an x402-compatible wallet, and the currently listed compatible wallets are all human-driven browser or mobile wallets. That gap — no programmatic, policy-governed wallet in the compatible set — is precisely the gap the DFNS layer fills, and it is a substantial part of why this project is worth funding.

4.2 Facilitator verification steps

The OpenZeppelin facilitator plugin, which implements the x402 v2 facilitator API for Stellar, performs the following on `/verify`: protocol validation (version, scheme, network), transaction XDR decoding, validation that the operation is an `invokeHostFunction` calling `transfer`, validation of amount and recipient against the stated requirements, validation that auth entries are present and signed by the payer, a check that the transaction envelope carries no signatures (so the relayer can rebuild it), a simulation to confirm the transaction will succeed, and a check that the transaction source is not the relayer itself.

On `/settle` it verifies again, extracts the operation and signed auth entries, submits via the relayer or an optional channel service, and waits for confirmation.

This is directly relevant to the contract design in Section 7: because the facilitator **simulates** the transaction before settling, any authorization failure in a Soroban payment-control contract surfaces as a clean pre-settlement rejection rather than a failed on-chain transaction. Spending-limit violations therefore cost nothing and return a deterministic error to the agent.

4.3 Facilitator options

Option	Networks	Notes
Build-on-Stellar Relayer with OpenZeppelin x402 plugin (hosted)	Testnet + Mainnet	Requires a Relayer API key; endpoints `https://channels.openzeppelin.com/x402/testnet` and `https://channels.openzeppelin.com/x402`
Coinbase x402 facilitator	Testnet only, sponsored fees	Useful as a cross-check during development
Self-hosted OpenZeppelin plugin	Testnet + Mainnet	Plugin published on npm and open-sourced; deployable on our own Relayer with our own asset configuration

Primary path: the hosted Build-on-Stellar facilitator, for both testnet and mainnet.

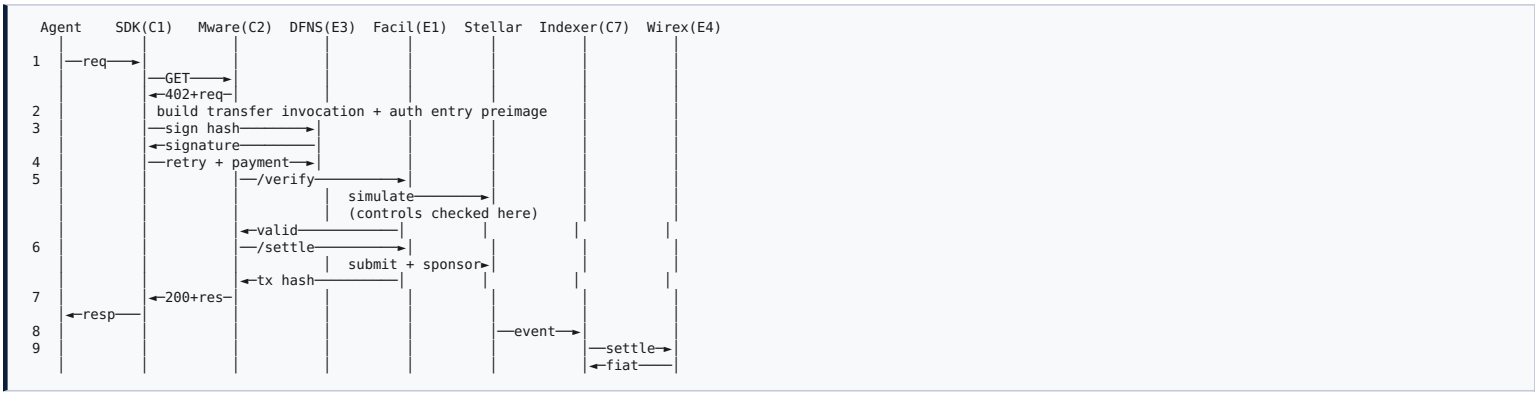
Documented fallback: self-hosting the same open-source plugin. This is not a theoretical fallback — it is the mitigation for the EURC question in Section 8, since a self-hosted facilitator lets us configure exactly which asset contracts are accepted.

The facilitator client (C3) is written against the x402 v2 facilitator API, not against a specific provider, so switching between these three options is a configuration change.

---

5. End-to-end payment flow

The following is the full mainnet flow, annotated with the component that performs each step.



Step detail

1 — Challenge. The middleware (C2) matches the route against its pricing configuration and returns `402` with the payment requirements for the configured network and asset.

2 — Authorization construction. The SDK (C1) resolves the asset's SEP-41 contract address, builds the `transfer` invocation, computes the authorization entry and its signature payload, and requests a signature from the wallet layer.

3 — Signing. The DFNS adapter (C4) requests a signature over the 32-byte authorization payload hash. DFNS policies are evaluated at this point (Section 6.3); a policy rejection returns a deterministic failure to the agent before anything reaches the network.

4 — Resubmission. The SDK attaches the payment payload and retries the original request. The transaction envelope is intentionally left unsigned so the facilitator can rebuild it.

5 — Verification. The middleware calls `/verify`. The facilitator simulates the transaction; the Soroban payment-control contract's authorization logic executes during simulation, so caps and allowlists are enforced here.

6 — Settlement. The middleware calls `/settle`. The facilitator submits the transaction and sponsors the network fee. Stellar's ~5-second finality means the merchant response is not meaningfully delayed.

7 — Response. The middleware validates the settlement result against the original requirements and serves the resource.

8 — Indexing. The indexer (C7), streaming from Stellar RPC/Horizon, records the payment independently of the request path. This is deliberate: reconciliation is derived from on-chain truth, not from application logs.

9 — Fiat settlement. The Wirex adapter (C9) maps each indexed on-chain payment to a fiat ledger entry through Wirex's API.

Idempotency and failure handling

Each payment carries a client-generated identifier propagated to the DFNS signature request as an idempotency key and stored by the indexer. The failure matrix:

Failure point	Behaviour	Financial risk
Policy rejection at signing	Deterministic error to agent	None — nothing signed
`/verify` rejection	402 returned with reason	None — nothing submitted
Simulation failure (cap exceeded)	402 with control-violation reason	None
`/settle` timeout, tx succeeded	Indexer detects payment; middleware reconciles on next poll	Resource may be served late, never double-charged
`/settle` failure, tx failed	Retry with same idempotency key	None
Merchant handler error after settlement	Payment recorded; refund flow via settlement registry	Handled at application level

---

6. Wallet and key management (DFNS)

6.1 Why a programmatic wallet layer is required

The x402-compatible wallets listed by Stellar today are human-driven. An autonomous agent making per-request micropayments cannot depend on a browser extension prompt. The wallet layer must therefore satisfy four requirements simultaneously: programmatic account creation at scale, signing without human interaction, policy enforcement before signing, and key custody that does not put Vottun or the merchant in possession of customer private keys.

DFNS was selected after evaluating alternatives. Browser-extension and connect-based wallet stacks were ruled out for requiring human interaction. DFNS provides MPC-based key management with a policy engine and an API-first model.

6.2 Account provisioning

For each agent and each merchant, the adapter (C4) creates a DFNS key/wallet on network `Stellar` (mainnet) or `StellarTestnet`, derives the Stellar account address, funds the minimum reserve from a platform-operated funding account, and establishes trustlines for the assets in use.

Wallets are addressed by their signing key: DFNS exposes the key identifier as `wallet.signingKey.id`, and signature requests are made against that key.

6.3 Signing model

The critical technical property is that DFNS supports **hash signing**: a signature request with `kind: "Hash"` over a 32-byte hex-encoded hash, documented as supported by all cryptographic schemes. Since a Soroban authorization entry is signed over a payload hash, this is the mechanism by which a DFNS-managed account participates in x402 — no Stellar-specific transaction type is required from DFNS, and the SDK retains full control over what is being signed.

DFNS additionally exposes Stellar as a first-class network and provides a `StellarFeeBumpTransaction` signing kind, which confirms Stellar-specific support beyond generic hash signing.

Signature requests move through an explicit lifecycle (`Pending → Executing → Signed → Confirmed`, or `Failed` / `Rejected`) and carry an `approvalId` and a policy-resolution timestamp — meaning the policy engine sits in the signing path by design, which is exactly where a spending control belongs for the off-chain enforcement path.

Two integration details that materially affect delivery, stated explicitly:

- **User Action Signing.** Change-inducing DFNS requests, including signature generation, require a user action signature in addition to the bearer token. For a non-interactive agent flow this must be configured as a service account or delegated signing arrangement rather than an interactive user credential. This shapes the T1 integration work and is called out here so it is not discovered late.
- **Policy granularity for hash signing.** DFNS documents that policies gate signature requests, but the public documentation does not specify whether a policy can discriminate by signature *kind* or inspect the contents of an opaque hash. This is an open question, listed in the risk register (Section 12, R3) and scheduled for confirmation with DFNS during the free 30-day sandbox period, before the D1.3 design is closed. The dual-path control design in Section 7.4 exists so that the project is not dependent on the answer.

6.4 Custody posture

DFNS operates key material under MPC; no single party holds a complete key, and Vottun does not hold customer keys. The formal responsibility split is set out in Section 11.

7. Soroban payment controls

7.1 Purpose

An enterprise will not let an autonomous agent spend from a funded account without bounds. The controls that matter in practice are: a maximum spend per unit of time per agent, a maximum per single payment, a restriction on which destinations may be paid, and a rate limit on payment frequency. These are the preconditions for enterprise adoption of agentic payments, and implementing them on Soroban rather than only in application code is what makes them verifiable by the party bearing the risk.

7.2 Contract inventory

`payment_controls` — holds the policy for each agent and authorizes or rejects payments.

```
// Administration
fn set_policy(env: Env, agent: Address, policy: Policy);
fn get_policy(env: Env, agent: Address) -> Policy;
fn pause(env: Env, agent: Address);          // emergency stop
fn resume(env: Env, agent: Address);

// Authorization
fn authorize_payment(
    env: Env,
    agent: Address,
    destination: Address,
    asset: Address,
    amount: i128,
) -> Result<(), ControlError>;

// Introspection (read-only, used by the dashboard)
fn spent_in_window(env: Env, agent: Address) -> i128;
fn remaining_allowance(env: Env, agent: Address) -> i128;

pub struct Policy {
    pub per_payment_max: i128,
    pub window_max: i128,
    pub window_seconds: u64,
    pub max_payments_per_window: u32,
    pub allowed_destinations: Vec<Address>,   // empty = unrestricted
    pub allowed_assets: Vec<Address>,
    pub paused: bool,
}
```

Storage layout: policies in persistent storage keyed by agent address; rolling-window counters in temporary storage with a TTL matched to the window, which keeps rent costs bounded and makes expiry self-cleaning.

`settlement_registry` — records settled payments and emits the events the indexer consumes.

```
fn record(env: Env, payment_id: BytesN<32>, agent: Address,
          merchant: Address, asset: Address, amount: i128);
fn get(env: Env, payment_id: BytesN<32>) -> Option<SettlementRecord>;
```

Events: `payment_authorized`, `payment_settled`, `control_violation`, `policy_updated`. Structured event topics give the indexer a stable contract rather than requiring transaction-body parsing.

7.3 Where enforcement happens

Enforcement occurs during authorization, which the facilitator triggers via simulation before submitting anything on-chain. A violation is therefore a free, deterministic rejection rather than a failed transaction.

7.4 The payer model — the project's principal technical risk

The clean design is for the agent's payer address to be a **Soroban custom account** (a contract account) whose `__check_auth` implements the control logic directly, with DFNS holding the signing key that `__check_auth` verifies. That makes the controls unbypassable, since no valid authorization can exist without passing them.

However: neither the Stellar x402 documentation nor the facilitator documentation states that a contract address is supported as the x402 payer. The facilitator validates that auth entries are "signed by the payer," which is unambiguous for a classic account and undocumented for a custom account. OpenZeppelin publishes a Smart Account implementation for Stellar, which suggests the primitive exists, but its acceptance by the facilitator's verification path is unverified.

This is stated plainly because pretending otherwise would be the wrong way to write an architecture document. The response is a **dual-path design**, resolved by a validation spike in the first two weeks of Tranche 1:

	Path A — on-chain	Path B — layered
Payer	Soroban custom account	Classic account (DFNS-managed)
Primary enforcement	`__check_auth` in `payment_controls`	DFNS policy engine, pre-signature
Secondary enforcement	—	`payment_controls` invoked by `settlement_registry` on the settlement path
Bypassability	Cryptographically impossible	Requires compromise of the platform signing path
Depends on	Facilitator accepting contract payers	Nothing undocumented

Path B delivers the same product behaviour with a weaker trust model, and it depends on nothing that is not already documented. **The tranche deliverables are written so that both paths satisfy them**, which means the risk cannot translate into a missed milestone. The spike outcome is a documented deliverable of D1.2 either way.

7.5 Audit and open-sourcing

Both contracts are released under an open-source license with verified mainnet deployments, documentation and usage examples. Where an audit is appropriate, the intended route is SDF's Soroban Security Audit Bank rather than the award budget, since audit costs are not an eligible SCF expense.

8. Asset strategy: USDC and EURC

x402 on Stellar supports any SEP-41 compliant token, with USDC as the default. Stellar publishes the USDC issuer and SEP-41 contract addresses for both testnet and mainnet, and those are the addresses the middleware resolves by default.

EURC requires explicit handling. The facilitator plugin is configured with an explicit list of accepted asset contract addresses per network. A hosted facilitator therefore accepts EURC only if EURC's contract is in its configuration. Since Wirex settles in both USDC and EURC, and the European merchant base is a core part of the go-to-market, EURC support is not optional for this product.

Approach:

1. Query the facilitator's `/supported` endpoint during the T1 spike to enumerate the accepted asset contracts on both networks.
2. If EURC is accepted by the hosted facilitator, no further work is needed.
3. If it is not, deploy a self-hosted instance of the open-source OpenZeppelin facilitator plugin on our own Relayer, configured with both the USDC and EURC contracts. The plugin is published on npm with a documented Docker Compose example; this is a configuration and deployment exercise, not new protocol work.

The middleware treats the asset as configuration throughout, so no application code changes between the two outcomes. Multi-asset pricing (a merchant quoting in EUR and accepting EURC, or quoting in USD and accepting USDC) is handled by the pricing configuration rather than by branching logic.

---

9. Indexing, reconciliation and the merchant dashboard

9.1 Indexer (C7)

The indexer streams contract events and payment operations from Stellar RPC and Horizon, deriving the merchant's payment history from network state rather than from the request path. It maintains a cursor with at-least-once delivery and idempotent writes keyed by transaction hash, so a restart or a stream gap causes reprocessing rather than loss.

Reconciliation runs as a three-way comparison: on-chain settled payments, middleware-recorded served requests, and Wirex fiat ledger entries. Discrepancies are surfaced as exceptions in the dashboard rather than silently corrected. This design choice exists because the finance team, not the engineering team, is the party that has to sign off on the numbers.

9.2 Dashboard (C8)

The dashboard is the surface through which the actual end user — a business, not a developer — operates the product: endpoint and pricing management, real-time payment monitoring, per-agent spending policy configuration (writing to `payment_controls`), balances, a transaction explorer linking to Stellar Expert, reconciliation exports (CSV and accounting-system formats), and the exception queue.

Every fiat line item is traceable to a Stellar transaction hash, and every Stellar transaction is traceable to an HTTP request. That bidirectional traceability is the auditability property finance teams require and is a first-class design goal, not a reporting afterthought.

---

10. Fiat settlement (Wirex) and the neutral gateway

10.1 Wirex settlement adapter (C9)

Wirex already operates live dual-stablecoin card settlement in USDC and EURC on Stellar, which is what makes the fiat leg an integration rather than a build. The adapter maps each indexed on-chain payment to a Wirex ledger entry, maintaining a per-transaction mapping in both directions, and exposes settlement status back to the dashboard. Delivery is staged: sandbox integration in Tranche 2, production integration with monitoring and alerting in Tranche 3.

10.2 Neutral gateway (C10)

The gateway normalizes x402 payment intents so that settlement always occurs on Stellar, regardless of where the paying agent or application originates. It issues Stellar-denominated payment challenges and routes settlement onto Stellar.

Explicitly excluded from scope: the gateway does not bridge value across chains, does not hold liquidity, and does not take FX risk. This is stated as a boundary rather than left to inference, because the alternative reading would imply a materially different — and unfunded — system. The gateway's purpose is to convert the product's neutrality into net-new settlement volume for Stellar.

---

11. Regulatory positioning

Responsibilities across the stack are explicit, and each maps to the party that holds the corresponding license or capability:

Party	Role	Regulatory posture
Wirex	Merchant onboarding, KYB/KYC, fiat balance and settlement rails	Regulated payments entity; the perimeter within which fiat is held and moved
DFNS	MPC key management infrastructure	Technology provider; keys are generated and used under MPC, not pooled customer funds
Vottun	Software layer — SDK, middleware, contracts, dashboard	Software provider; does not take possession of customer funds
SDF / OpenZeppelin	x402 facilitator, fee sponsorship	Infrastructure; submits transactions already authorized by the payer

The design property that matters under EU rules including MiCA: the merchant's crypto exposure exists inside a regulated partner's onboarding and settlement perimeter, and Vottun does not hold customer funds or the unilateral ability to move them. The "never holds crypto" merchant experience is delivered through that arrangement rather than by Vottun assuming a custodial role.

The final contractual allocation of these responsibilities is being confirmed with Wirex and DFNS. Nothing in this section describes a role that any party has not agreed to perform.

12. Risk register

ID	Risk	Likelihood	Impact	Mitigation	Owner
R1	Facilitator does not accept a Soroban custom account as payer	Medium	Medium	Dual-path design (§7.4); Path B depends on nothing undocumented; deliverables satisfied either way	Vottun
R2	EURC not in the hosted facilitator's accepted assets	Medium	Medium	Self-host the open-source facilitator plugin with EURC configured (§8)	Vottun
R3	DFNS policy engine cannot discriminate on hash-signing requests	Medium	Medium	Confirm during 30-day sandbox before closing D1.3; Path A enforcement is unaffected	Vottun / DFNS
R4	DFNS User Action Signing complicates non-interactive flows	Low	Medium	Service-account / delegated-signing configuration, addressed in T1 (§6.3)	Vottun
R5	Facilitator availability or rate limits in production	Low	High	Facilitator client is provider-agnostic; self-hosted Relayer as documented fallback	Vottun
R6	Wirex sandbox or production API timelines slip	Low	High	Sandbox in T2, production in T3, with a buffer; on-chain settlement functions independently of the fiat leg	Vottun / Wirex
R7	x402 specification evolves during the build	Medium	Low	Built against x402 v2; version negotiation via `/supported`	Vottun
R8	Soroban state archival affects control-window counters	Low	Low	Temporary storage with TTL matched to the policy window (§7.2)	Vottun

13. Non-functional design

Security. No private keys in application infrastructure; all signing through DFNS MPC. Facilitator and DFNS credentials in a managed secret store with rotation. Contract admin functions behind multi-signature control. Every settlement independently verified against on-chain state before a resource is served. Payloads validated against the original payment requirements to prevent replay across endpoints or merchants.

Environments. Development against Stellar testnet and the DFNS sandbox; staging against testnet with a production-shaped configuration; production on mainnet. Contract deployments are reproducible and version-pinned, with addresses published in the repository.

Observability. Structured logging with a correlation identifier spanning the HTTP request, the signature request, the facilitator call and the on-chain transaction. Metrics on payment latency, settlement success rate, control-rejection rate and reconciliation exceptions. Alerting on settlement failure rate, indexer lag and reconciliation drift.

Performance. Stellar's ~5-second finality and sub-cent fees are what make per-request micropayments economically viable; a per-request fee of a few cents would make the entire model uneconomic. The added latency in the request path is the signing round-trip plus verification and settlement. Where a merchant's latency budget cannot absorb settlement inline, the middleware supports serving on successful `/verify` with asynchronous settlement — a per-endpoint configuration choice with an explicit risk trade-off documented for the merchant.

Testing. Unit tests for middleware, SDK and contracts; Soroban contract tests covering caps, allowlists, velocity limits and pause behaviour; integration tests against testnet with a real facilitator; reproducibility validated by having an external developer complete the reference integration from published documentation alone.

14. Mapping to tranche deliverables

Tranche	Components delivered	Verification
T1 — MVP (wk 5, $22,000)	C1, C2, C3, C4 + payer-model spike	Core payment flow on testnet: 402 → auth-entry signature from a DFNS-managed wallet → facilitator settlement → resource served. Spike findings documented.
T2 — Testnet (wk 11, $33,000)	C5, C6, C7, C8, C9 (sandbox)	Full loop on testnet including the fiat leg; control contracts enforcing caps and allowlists in recorded demos; reference integration reproduced by an external developer.
T3 — Mainnet (wk 16, $44,000)	Mainnet deployment, C10, C9 (production), documentation, user testing	x402 payments settled on mainnet, verifiable on-chain; gateway routing live; fiat balance available through Wirex with documented reconciliation; contracts open-sourced with verified deployments.

15. References

Stellar - x402 on Stellar — https://developers.stellar.org/docs/build/agentic-payments/x402 - Signing Soroban invocations (auth-entry signing) — https://developers.stellar.org/docs/build/guides/transactions/signing-soroban-invocations - x402 on Stellar tooling and examples — https://github.com/stellar/x402-stellar - SEP-41 token interface — https://stellar.org/protocol/sep-41

Facilitator - OpenZeppelin x402 Facilitator guide — https://docs.openzeppelin.com/relayer/1.4.x/guides/stellar-x402-facilitator-guide - Facilitator plugin source — https://github.com/OpenZeppelin/relayer-plugin-x402-facilitator - OpenZeppelin Smart Account (Stellar) — https://docs.openzeppelin.com/stellar-contracts/accounts/smart-account

Wallet infrastructure - DFNS Generate Signature API — https://docs.dfns.co/d/api-docs/keys/generate-signature

Protocol - x402 specification — https://www.x402.org - Coinbase x402 repository — https://github.com/coinbase/x402

Prior delivered work - Wirex–Vottun–Stellar SDK (SCF-funded) — https://communityfund.stellar.org/project/wirex-vottun-stellar-sdk-a-scalable-development-framework-icl